

Variables & Control Structures

LEGAL NAME	<i>LEONARD-ADRIAN BUNEA</i>
PREFERRED NAME	ZOE BUNEA
TIME SPENT	01:23:55

Table of Contents

Exercise 1 – Rectangle Calculator	2
Solution Idea.....	2
Code.....	2
Tests	3
Exercise 2- FizzBuzz Game	4
Solution Idea.....	4
Code.....	4
Tests	5
Exercise 3 – Day of Week Processor	7
Solution Idea.....	7
Code.....	7
Tests	9
Exercise 4 – Star Pattern Generator	10
Solution Idea.....	10
Code.....	10
Tests	11
Exercise 5 – Collatz Conjecture.....	12
Solution Idea.....	12
Code.....	12
Tests	13
Exercise 6 – Dice Rolling Simulator	14
Solution Idea.....	14
Code.....	14
Tests	16

Exercise 1 – Rectangle Calculator

Solution Idea

Create methods to calculate each property and then print each of them out.

Code

```
public class RectangleCalculator 9 usages new *
{
    public double calculateArea(double width, double height) 2 usages new *
    {
        return width * height;
    }

    public double calculatePerimeter(double width, double height) 2 usages new *
    {
        return width * 2 + height * 2;
    }

    public double calculateDiagonal(double width, double height) 2 usages new *
    {
        return Math.sqrt(Math.pow(width, 2) + Math.pow(height, 2));
    }

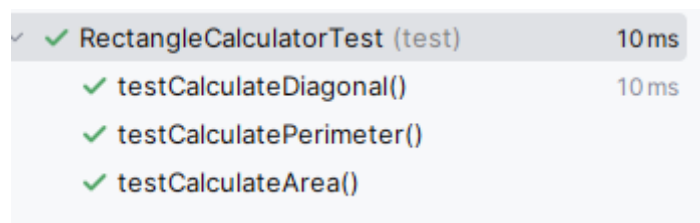
    public void calculateRectangleProperties(double width, double height) 1 usage new *
    {
        IO.println("=== Rectangle Calculator ===");
        IO.println("Width: \t" + width);
        IO.println("Height: \t" + height);
        IO.println("Area: \t" + calculateArea(width, height));
        IO.println("Perimeter: \t" + calculatePerimeter(width, height));
        IO.println("Diagonal: \t" + calculateDiagonal(width, height));
    }
}
```

Figure 1: RectangleCalculator

Tests

```
=== Rectangle Calculator ===  
Width: 5.0  
Height: 3.0  
Area: 15.0  
Perimeter: 16.0  
Diagonal: 5.830951894845301
```

Figure 2: Test Output



✓	✓ RectangleCalculatorTest (test)	10 ms
	✓ testCalculateDiagonal()	10 ms
	✓ testCalculatePerimeter()	
	✓ testCalculateArea()	

Figure 3: Unit Tests

Exercise 2- FizzBuzz Game

Solution Idea

Using if-statement, one can check whether a number is divisible by 5 or 3 and return the corresponding Fizz or Buzz. This can run inside a for-loop.

Code

```
public class FizzBuzz 5 usages ⓘ Zoe *
{
    private final int maxCount; 3 usages

    public FizzBuzz(int maxCount) 2 usages new *
    {
        this.maxCount = maxCount;
    }

    // returns the fizz buzz depending on the number
    public String getFizzBuzz(int number) 5 usages new *
    {
        if (number % 3 == 0 && number % 5 == 0)
        {
            return "FizzBuzz";
        }
        else if (number % 3 == 0)
        {
            return "Fizz";
        }
        else if (number % 5 == 0)
        {
            return "Buzz";
        }
        else
        {
            return Integer.toString(number);
        }
    }
}
```

Figure 4: FizzBuzz

Variables & Control Structures

```
// runs the FizzBuzz loop
public void runFizzBuzz() 1 usage new *
{
    IO.println("=== FizzBuzz Game ===");
    IO.println("Counting from 1 to " + maxCount);

    for (int i = 1; i <= maxCount; i++)
    {
        IO.println(getFizzBuzz(i));
    }
}
```

Figure 5: FizzBuzz

Tests

```
=== FizzBuzz Game ===
Counting from 1 to 15
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Figure 6: FizzBuzz Output

Variables & Control Structures

✓	✓ FizzBuzzTest (test)	8 ms
	✓ testGetFizzBuzz()	8 ms

Figure 7: FizzBuzz Unit Test

Exercise 3 — Day of Week Processor

Solution Idea

Create a validation method to check whether the day is between 1 and 7. Use a switch to determine name and an if to determine day type.

Code

```
public class DayOfWeek 5 usages  Zoe *
{
    public boolean isValid(int day) 5 usages  new *
    {
        if (day > 7 || day < 1)
        {
            return false;
        }

        return true;
    }

    public String getWeekDayName(int day) 10 usages  new *
    {
        // map days to day names using a switch statement
        switch (day)
        {
            case 1:
                return "Monday";
            case 2:
                return "Tuesday";
            case 3:
                return "Wednesday";
            case 4:
                return "Thursday";
            case 5:
                return "Friday";
            case 6:
                return "Saturday";
            case 7:
                return "Sunday";
        }

        return "Invalid";
    }
}
```

Figure 8: DayOfWeek

Variables & Control Structures

```
public String getDayType(int day) 6 usages new *
{
    if (day == 6 || day == 7)
    {
        return "Weekend";
    }
    else if (day >= 1 && day <= 5)
    {
        return "Weekday";
    }
    else
    {
        return "Invalid";
    }
}

public void checkDay(int day) 1 usage new *
{
    if (!isDayValid(day))
    {
        IO.println("Invalid day");
        return;
    }

    IO.println("=== Day of Week Processor ===");
    IO.println("Day Number: \t" + day);
    IO.println("Day: \t" + getWeekDayName(day));
    IO.println("Type: \t" + getDayType(day));
}
```

Figure 9: Day of Week

Tests

```
=== Day of Week Processor ===  
Day Number:      3  
Day:    Wednesday  
Type:    Weekday
```

Figure 10: DayOfWeek output

✓ DayOfWeekTest (test)	10 ms
✓ testGetWeekDayName_validDays()	9 ms
✓ testGetWeekDayName_invalidDays()	
✓ testIsDayValid_withValidDays()	1 ms
✓ testIsDayValid_withInvalidDays()	
✓ testGetDayType_invalid()	
✓ testGetDayType_weekdays()	
✓ testGetDayType_weekends()	

Figure 11: DayOfWeek Unit Tests

Exercise 4 — Star Pattern Generator

Solution Idea

A for loop inside another for loop.

Code

```
public class StarPattern 2 usages 3 Zoe *
{
    private final int patternSize; 3 usages

    public StarPattern(int patternSize) 1 usage new *
    {
        this.patternSize = patternSize;
    }

    public void generatePattern() 1 usage new *
    {
        IO.println("=== Star Pattern Generator ===");
        IO.println("Pattern size: " + patternSize);

        for (int i = 1; i <= patternSize; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                IO.print("* ");
            }

            IO.println("");
        }
    }
}
```

Figure 12: Star Pattern

Tests

```
=== Star Pattern Generator ===  
Pattern size: 5  
*  
* *  
* * *  
* * * *  
* * * * *
```

Figure 13: Star Pattern Output

Exercise 5 — Collatz Conjecture

Solution Idea

While the number isn't 1, with an if statement check whether it's divisible by two or not and do the appropriate calculation on the number.

Code

```
public class CollatzConjecture 5 usages 2 Zoe*
{
    // either divides by 2 or multiplies by 3 and adds 1
    public int processNumber(int number) 3 usages new *
    {
        if (number % 2 == 0)
        {
            return number / 2;
        }
        else
        {
            return number * 3 + 1;
        }
    }

    public int runCollatzConjecture(int startingNumber) 3 usages new *
    {
        IO.println("=== Collatz Conjecture ===");
        IO.println("Starting with " + startingNumber);

        int steps = 0;
        int number = startingNumber;

        while (number != 1)
        {
            number = processNumber(number);
            steps++;
        }

        IO.println("Reached 1 in " + steps + " steps!");

        return steps;
    }
}
```

Figure 14: Collatz Conjecture Code

Tests

```
=== Collatz Conjecture ===  
Starting with 5  
Reached 1 in 5 steps!
```

Figure 15: Collatz Conjecture Output

✓ CollatzConjectureTest (test)	10 ms	✓ 4 tests passed 4 tests total, 10 ms
✓ testProcessNumberOdd()	9 ms	C:\Users\cmdrp\.jdk\openjdk-25\bin\java.exe ...
✓ testRunCollatzConjectureFor1()	1 ms	=== Collatz Conjecture ===
✓ testRunCollatzConjectureFor6()		Starting with 1
✓ testProcessNumberEven()		Reached 1 in 0 steps!
		=== Collatz Conjecture ===
		Starting with 6
		Reached 1 in 8 steps!

Figure 16: Collatz Conjecture Unit Tests

Exercise 6 — Dice Rolling Simulator

Solution Idea

Generate random numbers in a while loop until random number is equal to target number.

Code

```
public class DiceRollingSimulator 5 usages ⓘ Zoe *
{
    // random number between 1 and 6
    public int rollDice() 2 usages new *
    {
        return (int)(Math.random() * 6) + 1;
    }

    public boolean isValidInput(int targetRoll) 3 usages new *
    {
        if (targetRoll >= 1 && targetRoll <= 6)
        {
            return true;
        }

        return false;
    }
}
```

Figure 17: Dice Rolling Sim

Variables & Control Structures

```
public int simulateRollsUntilTarget(int targetRoll) 3 usages new *
{
    if (!isValid(targetRoll))
    {
        IO.println("Invalid Input");
        return -1;
    }

    IO.println("=== Dice Rolling Simulator ===");
    IO.println("Target number " + targetRoll);

    int diceRoll = -1;
    int count = 0;

    while (diceRoll != targetRoll)
    {
        diceRoll = rollDice();
        count++;

        IO.println("Roll " + count + ": \t" + diceRoll);
    }

    IO.println("Found target number " + targetRoll + " in " + count + " rolls!");

    return count;
}
```

Figure 18: Dice Rolling Sim

Tests

```
=== Dice Rolling Simulator ===  
Target number 6  
Roll 1:    3  
Roll 2:    1  
Roll 3:    4  
Roll 4:    5  
Roll 5:    1  
Roll 6:    3  
Roll 7:    2  
Roll 8:    4  
Roll 9:    6  
Found target number 6 in 9 rolls!
```

Figure 19: Dice Roll Output

✓ ✓ DiceRollingSimulatorTest (test) 12 ms	✓ 5 tests passed 5 tests total, 12 ms
✓ testIsValidTrue() 11 ms	C:\Users\cmdrp\.jdk\openjdk-25\bin\java.exe ...
✓ testRollDiceRange()	=== Dice Rolling Simulator ===
✓ testIsValidFalse()	Target number 3
✓ testSimulateRollsUntilTargetValidInput() 1 ms	Roll 1: 2
✓ testSimulateRollsUntilTargetInvalidInput()	Roll 2: 5
	Roll 3: 4
	Roll 4: 1
	Roll 5: 5
	Roll 6: 6
	Roll 7: 6
	Roll 8: 3
	Found target number 3 in 8 rolls!

Figure 20: Dice Roll Unit Tests